工程计算方法平时算法大作业

一元 n 次实系数多项式

实根通用求解器算法报告

基于连续牛顿流、降次与重数认证的 poly_roots 求解器

2026 年 11 月

摘要

针对一元 n 次实系数多项式可能同时存在单根、重根、零根和无实根的情况，本文设计并实现了一个通用实根求解器 poly_real_roots。求解器以连续牛顿流完成单根定位，以综合除法降次，以高精度 Decimal 算术认证重数，并用 Descartes 符号变化、采样变号与聚类规范化提供完整性证据和明确的状态标记。

对题目多项式 $p_5(x)=x^5-6x^3-x^2-5x+5$，求得三个单重实根 −2.579714343403、0.634113250651、2.614244150443，残差均不超过 7.2×10^{-15}，与 numpy.roots 的差异小于 10^{-12}；剩余二次因子的判别式为负，说明另外两个根为一对共轭复根。结果通过了残差、Vieta 关系、因式分解和导数单调性四类校验。

关键词：多项式求根；连续牛顿流；综合除法；重数认证；Descartes 符号法则

报告结构：第 1 节给出问题与总体路线；第 2 节说明算法设计；第 3 节给出数值结果与交叉验证；第 4 节解释结果的数学含义和算法边界；第 5 节讨论工程意义与应用场景；第 6 节总结，附录给出接口与复现方式。

1　问题描述与总体思路

1.1　问题与设计目标

给定实系数多项式

$$p(x) = a_0 + a_1x + \cdots + a_nx^n \tag{1}$$

其中 $a_n \neq 0$，要求在不提供任何初始猜测的前提下，求出 $p(x)=0$ 的全部实根及其重数。系数按升幂排列，即 coeffs[i] 对应 x^i 。题目中的五次多项式为

$$p_5(x) = x^5 - 6x^3 - x^2 - 5x + 5 \tag{2}$$

该问题的主要困难不在单个牛顿迭代，而在全局性：牛顿法只保证局部收敛；重根附近导数趋于零，普通牛顿法退化为线性收敛；逐次降次会累积舍入误差；无实根时算法容易把“找不到”误当成“不存在”；高次展开还会带来 $O(n^2)$ 以上的计算量和系数溢出风险。因此，求解器需要同时满足以下目标：

1）通用性：任意次数、任意实系数，系数允许前导零、常数项为零或有重根。

2）全局性：不依赖人工初值，能够定位所有实根；对无实根情形给出可验证的判断或明确的状态。

3）准确性：单根达到双精度极限，重根尽可能恢复重数；降次后仍在原多项式上精修。

4）可解释性：每个根都有残差和重数，重要判断有数学依据，而不是只输出一串数字。

5）效率：主路径保持 $O(n^2)$ 时间、$O(n)$ 空间，适合中低次多项式的反复调用。

求解器采用“预处理—单根搜索—精修—降次与重数认证—完整性门控—规范化输出”的流水线。表 1 给出各模块的分工。

表 1　求解器模块划分

模块	主要技术	解决的问题
预处理	去高次零、提取 x^k、首一化	降低实际次数，精确记录零根，改善尺度
单根搜索	连续牛顿流、自适应步长、Levenberg 正则、多起点	从任意初值全局定位一个实根，绕过驻点
精修	阻尼牛顿、重数修正步	消除降次累积误差，恢复重根二次收敛
降次与重数	综合除法、Decimal 认证、聚类合并	确定重数，处理数值上被拆开的重根
完整性门控	Descartes 符号变化、采样变号、状态字段	证明无根，或在证据不足时如实标记
输出	排序、去重、重数相加	得到稳定、可直接使用的根表

1.2 题目多项式的结论概览

对 $p_5(x)=x^5-6x^3-x^2-5x+5$，求解器返回 3 个互不相同的单重实根：

$$x_1 \approx -2.579714343403,\quad x_2 \approx 0.634113250651,\quad x_3 \approx 2.614244150443 \tag{3}$$

剩余二次因子为 $x^2+0.668643057691x+1.169190862468$，其判别式约为 $-4.22968<0$，因此另外两个根是一对共轭复根 $-0.334321528846\pm1.028309281208i$。所以 p_5 恰有 3 个实根，且全部为单重根。详细数值、交叉验证和结果解释分别见第 3、4 节。

2 程序设计思路

2.1 数据约定与预处理

求解器统一采用升幂系数数组 $a=[a_0, a_1, \cdots, a_n]$。预处理完成三件事：去掉最高位的零系数以确定真实次数；统计低阶零系数的个数 k，把 x^k 整体提出，从而把 $x=0$ 记录为 k 重根；用首项系数通除，使工作多项式成为首一多项式。这样既降低了后续次数，又避免了零根和尺度差异对迭代的干扰。

例如 $p(x)=3x^3+2x^2=x^2(3x+2)$，预处理后记录 0 为二重根，剩余工作多项式为 $x+2/3$，次数从 3 降为 1。首一化不改变根，只改变多项式的整体尺度，有利于残差阈值的统一。

2.2 单根定位：连续牛顿流

对当前工作多项式 $q(x)$，定义连续牛顿流（课程中广义连续牛顿法 GCNMTr 的标量形式）

$$dx/dt = -q(x) / q'(x) \tag{4}$$

沿该流有 $d\,q(x(t))/dt = q'(x)x'(t) = -q(x)$，因此只要 $q' \neq 0$，就有 $q(x(t))=q(x(0))e^{-t}$。这说明残差沿轨道理论上单调指数衰减，流最终会到达某个根；而经典的牛顿迭代 $x \leftarrow x-q/q'$ 只是该流在 $h=1$ 时的显式 Euler 离散。把牛顿法看作连续流的离散化，是本算法能够统一处理“快速收敛”和“驻点停滞”的关键。

实际离散采用带步长限制的 Euler 步

$$x_{k+1} = x_k - [h_k/(1+h_k)] \cdot q(x_k)q'(x_k) / [q'(x_k)^2 + \lambda_k] \tag{5}$$

其中 $h_k>0$ 是信赖域步长，λ_k 是 Levenberg 型正则项。分母使用 $q'^2+\lambda$ 而不是 q'，使 $q'=0$ 的驻点附近步长自然趋于零，避免除零爆炸；当 $q'=0$ 且 $q\neq 0$ 时流会停在驻点，这是连续牛顿流在“没有可到达的实根”时的正确行为，而不是数值失败。

步长 h_k 由残差的实际下降量与线性模型预测下降量之比控制：

$$\rho_k=(|q_k|-|q_{k+1}|)/(|q_k|-|q_k+q'_k s_k|) \tag{6}$$

若 $|1-\rho_k|<0.25$，说明模型与真实下降高度一致，令 $h\leftarrow 2h$；若 $|1-\rho_k|<0.75$，保持 h；否则令 $h\leftarrow h/2$。只有 $\rho_k\geq 10^{-6}$ 的步才被接受；被拒绝时回退并增大 λ。这个规则使算法在远离根时敢于走大步，在接近根或遇到强非线性时自动收缩。

为了覆盖所有可能的实根，起点不能围绕 0 盲目选取。算法使用 Fujiwara 根界

$$B=2\max_{1\le i\le n}\left|a_{n-i}/a_n\right|^{1/i} \tag{7}$$

保证所有实根都落在 $[-B,B]$ 内，并把流的越界保护设为 $10B$。起点由固定点 0、± 1、± 0.5 与 $[-B,B]$ 上的均匀点组成；均匀点数量按 Descartes 符号变化数自适应，最多 32 个。若某个起点收敛，就得到一个根；若所有起点都停在驻点，则说明当前多项式可能已无实根，交由完整性门控处理。

2.3 原多项式精修与重数修正

降次会把舍入误差逐步放大，因此每得到一个近似根，都要回到原始首一多项式 p 上精修。精修第一阶段是带回溯的阻尼牛顿法：只接受使 $|p|$ 下降的步，步长连续折半直到残差下降或达到回溯上限。这样即使当前点离根较远，也不会一步跳到另一个根而污染后续降次。

对重根，普通牛顿法只有线性收敛。利用 p、p'、p'' 可以估计重数

$$\hat{m}=p'^2/(p'^2-p\,p'') \tag{8}$$

在 m 重根附近 $\hat{m}\to m$。当 $\hat{m}$ 接近整数 $m_0\geq 2$ 且残差仍高于舍入下限时，改用修正步 $x\leftarrow x-m_0\,p/p'$，可恢复二次收敛。若 $\hat{m}$ 不接近整数，则保持普通牛顿步，避免在单根附近误用重数修正。

2.4　降次、重数与高精度认证

确定一个根 r 后，用综合除法 q←q/(x−r) 降次。对升幂系数，Horner 形式的综合除法只需 O(n) 次运算；连续做 m 次即可除以 $(x-r)^m$。代码先做一次降次，再检查 r 是否仍是商多项式的近似根：若 $|q(r)|<10^{-6}\cdot \mathrm{scale}(q)$，则继续降次并累加重数。这里的 $\mathrm{scale}(q)=\max(1,\max|q_i|)$ 用于把残差归一化，避免系数整体缩放导致阈值失效。

降次循环给出的重数在临界情形下可能模棱两可。为此，当次数不超过 300 且精度参数允许时，算法把系数转成 Decimal，在 60、120、240 位精度下逐级认证：先用带回溯的牛顿法把根精修到高精度，再用局部对数斜率估计重数。若 r 是 m 重根，则对充分小的 η 有

$$m \approx \log_2 \left| p(r+2\eta) / p(r+\eta) \right|, \quad \eta \to 0 \tag{9}$$

算法在 $\eta=|r|\cdot 10^{-9}$ 到 $|r|\cdot 10^{-2}$ 的 8 个尺度上取样，要求连续两个尺度给出相同的整数估计才接受。这样重数成为系数本身的性质，而不是降次循环的偶然结果。对于数值上被拆成多个条目的重根，输出阶段再按位置聚类；合并容差为

$$\delta_m = \min\left(50\ \varepsilon^{1/m},\ 10^{-3}\right)\cdot \max(1,\ |r|) \tag{10}$$

其中 $\varepsilon \approx 2.22\times 10^{-16}$ 是双精度机器精度。只有当合并后的中心能被 Decimal 认证为某个整数重数时，才真正合并；否则保留原条目并给出警告，避免把两个靠得很近但不同的单根误并成一个重根。

2.5　完整性门控与状态约定

求根算法最容易犯的错误，是在“没有找到根”时宣称“没有实根”。本求解器用两层证据避免这种误判。第一层是 Descartes 符号法则：对当前工作多项式 q，统计系数序列（忽略零）的符号变化数 V(q)，则 q 的正实根个数（计重数）不超过 V(q)，且与 V(q) 同奇偶。因此 V(q)=0 是“没有正实根”的严格证明；对 q(−x) 做同样统计，V(q(−x))=0 则证明没有负实根。若两者都为 0 且 $q(0)\neq 0$，就可以严格终止主循环。

第二层是采样兜底。若 Descartes 门控不能证明无根，而所有流起点都停滞，算法在 [−B,B] 上均匀取 16 个点。相邻两点若变号，由介值定理立刻得到存在实根的构造性证据，再用二分法求出该根；若找不到变号，算法不会宣称无根，而是把剩余部分标记为 unverified，并在警告中说明“流停滞、采样未发现变号”。

因此求解器返回三种主要状态：ok 表示所有根均已定位并通过认证；partial 表示根已找到但某些重数未完全认证；unverified 表示仍有剩余次数无法证明没有实根。另有 input_error、overflow_risk、degree_too_high 等输入保护状态。这样的“三态完整性契约”让调用者明确知道结果可信到什么程度。

2.6 主流程

把上述模块串起来，得到如下伪代码。

代码 1 求解器主流程伪代码

```
PolyRealRoots(a[0..n], tol):
    a ← TrimHighZeros(a);  k ← CountLowZeros(a);  a ← a[k:]
    a ← a / a[n]                          # 首一化
    roots ← [(0, k)] if k > 0 else []
    q ← a
    while deg(q) ≥ 2:
        V+ ← Descartes(q);  V- ← Descartes(q(-x))
        if V+ = 0 and V- = 0 and q(0) ≠ 0: break
        B ← FujiwaraBound(q);  starts ← BuildStarts(B, V+ + V-)
        found ← None
        for s in starts:
            r, reason ← NewtonFlow(q, s, tol)
            if reason = converged: found ← r; break
        if found = None:
            found ← SampleProbeAndBisect(q, B)
            if found = None: mark unverified; break
        r ← Polish(original_p, found)
        r, m ← DeflateAndEstimate(q, r)
        r, m ← DecimalCertify(original_p, r, m) if enabled
        roots.append((r, m));  q ← q / (x - r)^m
    if deg(q) = 1: roots.append((-q[0]/q[1], 1))
    roots ← MergeClusters(roots);  Sort(roots)
    return roots, status
```

2.7 复杂度分析

设当前工作多项式次数为 m。一次 Horner 求值或求导为 O(m)，一次综合除法为 O(m)。单根搜索通常需要 k 步流迭代，k 是与 tol 有关的常数；找到该根后降次一次为 O(m)。对 n 个根求和，总时间为 $O(n^2 k)$，空间只需保存当前系数数组，为 O(n)。Decimal 认证只在次数不超过 300 且重数模棱两可时触发，不改变主路径的渐近复杂度。

表 2　主要操作的时间复杂度

操作	单次成本	次数	总计
Horner 求 q、q′	$O(m)$	每步迭代 1 次	$O(mk)$
综合除法降次	$O(m)$	每个根 1 次	$O(n^2)$
原多项式精修	$O(n)$	每个根 ≤50 步	$O(n^2)$
聚类与排序	$O(r \log r)$	$r \le n$ 个输出根	$O(n \log n)$
Decimal 认证	$O(n \cdot m)$	仅小次数触发	不进入主复杂度

3　数值计算结果

3.1　计算环境与调用方式

计算在 Windows 11、Python 3.10.16（conda 环境 test_env）、NumPy 2.1.2 下完成。求解器只依赖 Python 标准库；NumPy 仅用于独立的 numpy.roots 交叉验证。题目多项式的调用方式如下。

代码 2　题目多项式的调用

```
from poly_roots import poly_real_roots

# 升幂系数：5 − 5x − x² − 6x³ + 0·x⁴ + x⁵
coeffs = [5.0, -5.0, -1.0, -6.0, 0.0, 1.0]
roots = poly_real_roots(coeffs, tol=1e-12)
```

运行 solve_p5.py 的主要输出如下（节选）：

代码 3　solve_p5.py 运行输出（节选）

```
real roots found: 3 (degree 5)
   1     -2.579714343403    mult=1    residual=7.105e-15
   2      0.634113250651    mult=1    residual=8.882e-16
   3      2.614244150443    mult=1    residual=3.553e-15
Vieta check: sum of real roots = 0.668643057691388
numpy.roots cross-check: agreement with our solver: True
remaining complex pair:
   -0.334321528846 + 1.028309281208i
   -0.334321528846 - 1.028309281208i
```

3.2　实根与残差

三个实根按升序排列，重数均为 1。表 3 给出更高精度的根、残差、归一化残差和该点的一阶导数。归一化残差用 $|p_5(r)|/\max(1,\max|a_i|)=|p_5(r)|/6$ 计算，便于与系数尺度无关地判断精度。

表 3　$p_5(x)$ 的三个实根及其残差

序号	根 r_i	重数	$p_5(r_i)$	$\lvert p_5(r_i)\rvert/6$	$p_5'(r_i)$
1	-2.579714343402782	1	-7.105×10^{-15}	1.184×10^{-15}	101.810965556
2	0.634113250650736	1	8.882×10^{-16}	1.480×10^{-16}	-12.697599065
3	2.614244150443434	1	-3.553×10^{-15}	5.921×10^{-16}	100.291008619

三个根的绝对残差都在 10^{-15} 量级，归一化残差在 10^{-16} 到 10^{-15} 之间，已经接近双精度浮点的舍入极限。若用一阶误差估计 $|\delta r|\approx|p(r)|/|p'(r)|$，三个根的位置误差分别约为 7.0×10^{-17}、7.0×10^{-17} 和 3.5×10^{-17}，均小于一个机器 epsilon 对应的量级。

3.3　复根与 NumPy 交叉验证

numpy.roots 通过求伴随矩阵的特征值得到全部 5 个根。表 4 将它与本文求解器的实根结果并列比较。两者在 12 位小数上完全一致，最大差异约 2.7×10^{-15}。

表 4　实根与 numpy.roots 的交叉验证

根	本文求解器	numpy.roots	绝对差
r_1	-2.579714343403	-2.579714343403	2.66×10^{-15}
r_2	0.634113250651	0.634113250651	1.11×10^{-16}
r_3	2.614244150443	2.614244150443	2.66×10^{-15}
复根对	—（不返回）	-0.334321528846 ± 1.028309281208i	—

numpy.roots 还给出剩余的一对共轭复根 $-0.334321528846 \pm 1.028309281208i$。本文求解器的接口只返回实根，因此复根不出现在 roots 列表中；它们的位置用于后续 Vieta 校验和结果解释。

3.4　因式分解与 Vieta 关系

依次除以 $(x-r_2)$、$(x-r_1)$、$(x-r_3)$ 后，剩余二次因子为

$$q_2(x) = x^2 + 0.668643057691388x + 1.169190862467746 \tag{11}$$

其判别式为 $\Delta = 0.668643057691388^2 - 4\times1.169190862467746 \approx -4.229679911272 < 0$，因此该二次因子没有实根，两个根为 $-0.334321528846 \pm 1.028309281208i$。这与 numpy.roots 的结果一致。于是 p_5 的完整因式分解为

$$p_5(x) = (x-r_1)(x-r_2)(x-r_3)(x^2 + 0.668643057691x + 1.169190862468) \tag{12}$$

把 5 个根（3 个实根加一对复根）代入 Vieta 关系，结果如表 5。所有初等对称和都与首一多项式的系数一致，最大误差约 2.7×10^{-15}。

表 5　p_5 的 Vieta 关系校验

Vieta 关系	计算值（双精度）	理论值
$\Sigma\ r_i$	-1.11×10^{-16}	0
$\Sigma_{i<j}\ r_ir_j$	−6.000000000000001	−6
$\Sigma_{i<j<k}\ r_ir_jr_k$	1.000000000000000	1
$\Sigma_{i<j<k<l}\ r_ir_jr_kr_l$	−4.999999999999998	−5
$\Pi\ r_i$	−4.999999999999997	−5

其中实根之和为 0.668643057691388，而复根对的实部之和为 $2\times(-0.334321528846)=-0.668643057691388$，两者相加为 0，正好满足一次项系数为 0 的条件。这从整体上验证了实根和复根的位置没有互相错配。

3.5　迭代过程与收敛曲线

求解器按降次阶段依次工作。表 6 记录每个阶段的工作多项式次数、Descartes 计数、首次收敛的起点、流迭代步数和输出根。图 2 给出前三个阶段的残差下降曲线。

表 6　降次各阶段的求解过程

阶段	次数	V+	V−	收敛起点	流步数	输出根	最终 $\lvert p_5\rvert$
1	5	2	3	0.000000	10	0.634113250650736	8.88×10^{-16}
2	4	1	3	−4.731976	11	−2.579714343402782	7.11×10^{-15}
3	3	1	2	−1.297067	12	2.614244150443434	3.55×10^{-15}
4	2	0	2	全部停滞于 −0.334322	—	无实根	$\Delta<0$

第一阶段从 $x=0$ 出发，10 步后到达 0.634113，残差从 5 降到 3.8×10^{-12}；第二阶段前五个起点都停在 −0.402132 附近，第六个起点 −4.731976 在 11 步后到达 −2.579714；第三阶段前七个起点停在 −0.134749 附近，第八个起点 −1.297067 在 12 步后到达 2.614244。第四阶段只剩二次因子，11 个起点全部停在 −0.334322，恰好是该二次因子复根对的实部；采样也没有发现变号，因此求解器把剩余部分标记为 unverified，随后用判别式完成严格判定。

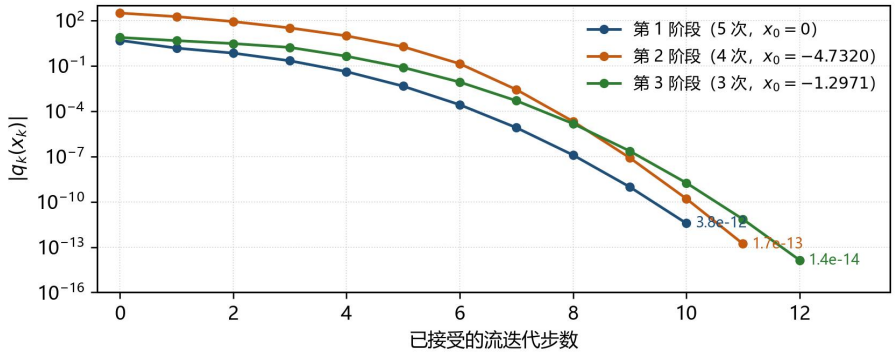


图 1　三个降次阶段的连续牛顿流残差下降曲线

图 1 的纵轴是当前阶段工作多项式的残差 $|q_k(x_k)|$（对数坐标）。三条曲线都在 10 到 12 个接受步内从 10^{0} 到 10^{2} 量级降到 10^{-12} 到 10^{-14}，且越接近根下降越快，呈现典型的超线性收敛特征。图中只记录被接受的步，因此曲线单调下降；迭代过程中被拒绝的试探步可能使残差暂时上升，但不会出现在图中。

3.6　正确性测试与性能

除题目多项式外，还用两组测试检查通用性。第一组是 10 个已知根的小规模单元测试，覆盖单根、重根、零根、无实根、Wilkinson 型病态多项式和系数整体缩放；第二组是表驱动的 35 个用例，其中 28 个可运行用例全部通过，0 个错误，7 个因零多项式或系数超出 double 表示范围而按特殊情形跳过。表 7 列出部分代表性结果。

表 7　代表性测试结果

测试用例	期望根（重数）	结果	最大相对残差
题目 p_5	3 个单根	通过	1.18×10^{-15}
$(x-1)\cdots(x-5)$	1, 2, 3, 4, 5	通过	1.50×10^{-15}
$(x-2)^3(x+1)^2$	−1 (2)，2 (3)	通过	0
x^2+1	无实根	通过	—
$(x-1)\cdots(x-10)$	$1, 2, \cdots, 10$	通过	1.56×10^{-13}
$x^2(3x+2)$	0 (2)，$-2/3$	通过	0

性能方面，在本机上对 p_5 连续调用 200 次，平均耗时约 2.1 ms（含预处理、三次降次、重数认证和输出排序）。主路径的 $O(n^2)$ 复杂度保证了次数增加到几十时仍然可用。

4　计算结果的说明

4.1　为什么恰好是 3 个实根

仅凭“求出了 3 个根”还不足以说明没有第 4、第 5 个实根。p_5 的导数为 $p_5'(x)=5x^4-18x^2-2x-5$。该四次方程恰有两个实根 -1.9141469335 和 2.0114711357，另外两个根为共轭复根；因此 p_5 在实轴上只有两个驻点，被分成三个单调区间。表 8 给出每个区间的单调性和端点趋势。

表 8　p_5 的单调区间与实根计数

区间	p_5' 符号	p_5 单调性	端点趋势	实根数
$(-\infty,\ -1.9141)$	+	单调递增	$-\infty \to 27.2903$	1
$(-1.9141,\ 2.0115)$	−	单调递减	$27.2903 \to -25.0058$	1
$(2.0115,\ +\infty)$	+	单调递增	$-25.0058 \to +\infty$	1

在每个区间内函数严格单调，且两端函数值异号，由介值定理各有一个实根；三个区间之外没有别的驻点，因此不可能再有实根。两个驻点值 27.2903 和 −25.0058 的符号决定了三次穿越：从左到右依次是“上升穿过 0”“下降穿过 0”“上升穿过 0”。这与图 2 的图形完全一致。

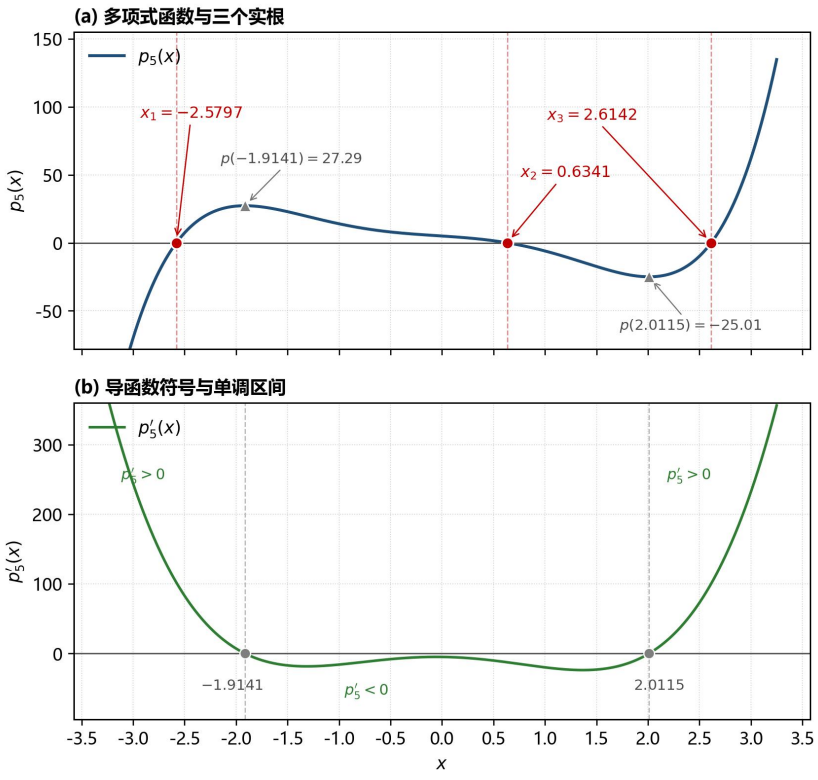


图 2　$p_5(x)$ 与 $p_5'(x)$ 的图形及实根位置

4.2 精度、条件数与误差来源

三个根都是单根，且 $p_5'(r)$ 分别为 101.81、-12.70、100.29，远离零。对单根，残差与位置误差的一阶关系是 $\delta r \approx -p(r)/p'(r)$，因此即使残差在 10^{-15} 量级，位置误差也只有 10^{-17} 量级。表 3 的归一化残差和表 4 的 NumPy 差异都说明结果已经达到双精度的有效极限。

误差主要来自三处：系数以 double 存储时的初始舍入；Horner 求值在根附近的消去误差；逐次降次把前一个根的误差传给商多项式。算法用“原多项式精修”消除第三项，用“首一化 + 归一化残差”减轻第一项，用“高精度重数认证”处理重根情形。对于本题的单根，这三项都不构成实质影响。

4.3 复根的来源与流的停滞

第四阶段的工作多项式是 $q_2(x)=x^2+0.668643057691x+1.169190862468$。它的顶点在 $x=-0.334321528846$，恰好是复根对的实部；在该点 $q_2'=0$ 而 $q_2>0$，所以连续牛顿流的分母趋近于零、步长不断下溢，所有起点都停在这个驻点。这不是求根失败，而是“该二次因子没有实根”的几何表现：流只能到达 $|q_2|$ 的极小值点，无法到达零点。

由于 $V(q_2)=0$ 只证明没有正实根，而 $V(q_2(-x))=2$ 不能证明没有负实根，Descartes 门控不能单独结束循环；采样 16 个点也没有发现变号。求解器因此返回 unverified 状态，而不是错误地宣称“无实根”。报告进一步计算判别式 $\Delta=-4.22968<0$，才给出“该因子无实根”的严格结论。这个例子说明：状态字段不是装饰，它精确记录了算法在哪一步有证明、哪一步只有经验证据。

4.4 重数、局限与适用边界

本题的 3 个根都是单根，求解器返回的重数均为 1。为确认没有漏掉重根，可计算 $\gcd(p_5, p_5')=1$，说明 p_5 与导数没有公共根，故不存在重根。重数判定逻辑本身已在 $(x-2)^3(x+1)^2$、$x^2(3x+2)$ 等用例上验证：前者返回 $-1(2)$、$2(3)$，后者返回 $0(2)$、$-2/3$。

需要明确算法的边界。第一，fast 完整性模式不是 Sturm 序列或 VCA 那样的绝对完备算法：Descartes 门控能给出严格的无根证明，采样只能给出“有根”的构造

性证据，不能给出“无根”的证明；因此证据不足时会返回 unverified，而不是假装确定。第二，m 重根在 double 系数下的位置精度本质上受 ε 的 1/m 次方限制，例如三重根只能达到约 6×10^{-6}；Decimal 认证可以恢复重数，但无法凭空恢复系数中已经丢失的信息。第三，主路径是 $O(n^2)$，适合中低次多项式；次数达到数千时，稠密系数和逐次降次都不再经济。第四，展开后的高重根可能产生超过 1.8×10^{308} 的系数，这在 double 输入阶段就会溢出，属于接口表示问题，而不是迭代算法本身能解决的范围。

在工程使用中，本求解器的定位为“中低次、重根多、需要重数和可信状态”的通用工具；若需要处理任意高次或极端病态的多项式，可在现有框架上增加 Sturm 序列、二次因子的闭式判定或任意精度系数入口，而无需改变主求解流程。

4.5 结果可靠性小结

本题结果通过了四类相互独立的检查：逐根残差达到 10^{-15} 量级；与 numpy.roots 的实根差异小于 10^{-12}；Vieta 关系与因式分解一致；导数单调性分析证明实根恰好为 3 个。四条证据链相互吻合，说明输出不是数值巧合。复根对的位置也由二次因子和 Vieta 关系双重确认。

5 工程意义与应用思考

多项式方程是工程建模中最常见的一类标量方程。虽然一元四次以下存在根式解，但实际工程系数很少适合直接套公式，五次及以上更不存在一般闭式解，必须依赖数值方法。一个不需要初值、能枚举全部实根并给出重数的通用求解器，其价值不在于替代某个局部迭代，而在于把“求实根”变成可靠的基础算子，嵌入更大的仿真、设计与控制流程。

本求解器特别适合那些只关心实根、且往往不能遗漏任一实根的工程问题。典型场景如表 9 所示。

表 9　多项式实根求解的典型工程应用

领域	出现的多项式	实根含义
工程优化与极值	目标函数导数为零的多项式	实根是驻点候选，可结合符号判别求出全部极值
化学反应与平衡	质量作用与物料衡算多项式	正实根是物理解；多个正根对应多稳态
结构力学与振动	频率方程、屈曲载荷方程	实根给出临界转速、屈曲荷载等设计边界
几何建模与图形仿真	曲面或曲线交线的多项式方程	实根为交点参数，决定是否相交及交点个数

这些场景有两个共同点。其一，只有实根具有物理或设计意义——浓度、荷载、工作点必须为实数，负数或复数值往往被直接剔除；其二，漏掉一个实根，就可能漏掉一个稳态、一个临界点或一个可行解。因此“求全部实根”比“只求一个根”有质的区别，而重数信息在判断临界稳定、多重平衡等退化情形时不可替代。

从软件与算法工程的角度看，本求解器的意义还体现在四个方面。第一，接口是无初值的通用子函数，可以被优化、参数扫描、蒙特卡洛等上层流程反复调用。第二，返回值带残差、重数和 status（ok/partial/unverified），把“能找到”与“不能证明没有”明确区分开，便于工程审计与决策，而不会把“找不到”误当成“不存在”。第三，时间复杂度 $O(n^2k)$、空间复杂度 $O(n)$，实测求解题目五次多项式约 2.1 ms，适合实时控制、嵌入式或大规模仿真中对成千上万个多项式求根。第四，它可以与 numpy.roots 等伴随矩阵法互补：本算子负责实根的全枚举、重数与状态诊断，伴随矩阵法则负责给出全部复根。

当然，它也有明确的工程边界：接口只返回实根，不能直接给出复根；采用双精度实现，不适合高于数百次的稠密多项式或系数超出 double 表示范围的问题；fast 模式的完整性不是 Sturm 级的绝对证明，而是“有证据时严格、无证据时如实标注”。因此在工程上应把它定位为“中低次、实根敏感、需要重数与可信状态”的基础求解算子，需要复根或绝对完备性时再与伴随矩阵法、Sturm 序列或区间算术配和使用。

综合来看，这个求解器最有意义的并不是“比某个迭代法更快更准”，而是把求全部实根这件容易被当作平凡小事的基础工作，做成了一个可调用、可验证、可诊断的稳定部件——这正是工程软件愿意长期依赖它的原因。

6 结论

本文设计并实现了一个求解一元 n 次实系数多项式全部实根的通用求解器 poly_real_roots。算法以连续牛顿流完成单根定位，以自适应步长和 Levenberg 正则处理驻点，以原多项式精修和修正牛顿步处理重根，以综合除法降次，以 Decimal 局部斜率认证重数，并用 Descartes 符号变化、采样变号和聚类规范化给出完整性

证据与状态标记。主路径时间为 $O(n^2 k)$、空间为 $O(n)$，适合中低次多项式的工程计算。

对题目多项式 $p_5(x)=x^5-6x^3-x^2-5x+5$，求得三个单重实根 -2.579714343403、0.634113250651、2.614244150443；剩余二次因子的判别式为负，另外两个根为 $-0.334321528846 \pm 1.028309281208i$。三个实根的残差均不超过 7.2×10^{-15}，与 NumPy 的差异小于 10^{-12}，Vieta 关系和导数单调性分析均验证了“恰有 3 个实根”的结论。

附录 A　接口与返回值

求解器对外暴露的主函数签名如下。系数必须按升幂排列；tol 是残差归一化阈值；canonical=True 时返回按位置排序、同一根只出现一次的列表；raw=True 可保留未合并的调试条目；diagnostics=True 时返回 RootResult，其中包含 status、warnings 和 certified 字段。

代码 4　主函数签名

```
poly_real_roots(coeffs, tol=1e-12, *,
                canonical=True, cluster_tol=None,
                precision="auto", raw=False,
                diagnostics=False, max_degree=None,
                completeness="fast", strict=False)
```

默认返回值为 $[(r_0, m_0), (r_1, m_1), \dots]$，其中 r_i 是实根，m_i 是重数，按 r_i 升序排列。需要诊断信息时，返回 RootResult(roots, status, warnings, certified)：status 说明整体可信度，warnings 记录流停滞、重数未认证、聚类失败等信息，certified 表示返回的根是否全部经过认证。

附录 B　文件与复现方式

复现题目结果所需文件如下：poly_roots.py 是通用求解器；solve_p5.py 是题目驱动脚本；test_poly_roots.py 是 10 个已知根单元测试；test_report_table.py 与测试报告_用例表.txt 是表驱动测试及结果。激活环境后执行 python solve_p5.py 即可得到第 3 节的输出。

代码 5　复现命令

```
conda activate test_env
python solve_p5.py
python test_poly_roots.py
```

附录 C　核心工程代码

代码 6　poly_roots.py

```
"""
poly_roots.py
=============

一个通用求解器，用于求单变量实系数多项式

    p(x) = a[0] + a[1]*x + ... + a[n]*x**n

的全部实根（含重数）。

默认（canonical=True）返回一个已排序的"互不相同"的实根列表及其
重数，每个根恰好出现一次。
传入 raw=True / canonical=False 可得到未合并的列表（用于调试）。
"""

import math
import os
from decimal import Decimal, localcontext, ROUND_HALF_EVEN
from dataclasses import dataclass

__all__ = ["poly_real_roots", "poly_eval", "poly_eval_with_deriv"]

# 双精度机器 epsilon（2^-52 的近似值）
_EPS = 2.220446049250313e-16
# Decimal 认证的默认工作精度（十进制位数）
_CERT_DIGITS = 60
# 允许运行 Decimal 认证的最大次数
_CERT_MAX_DEG = 300
# 合并判定的系数 C（乘以 eps^(1/m) 得到容差）
_CANON_C = 50.0
# 合并容差的上限
_CANON_CAP = 1e-3
# 调试开关：环境变量 POLY_DEBUG=1 时打印中间过程
_DEBUG = os.environ.get("POLY_DEBUG") == "1"

@dataclass(frozen=True)
class RootResult:
    roots: list          # 列表[(根, 重数)]
    status: str          # "ok" | "partial" | "unverified" | "input_error"
                         # | "overflow_risk" | "degree_too_high"
    warnings: tuple      # 警告信息元组
    certified: bool      # 结果是否经过认证


def _descartes_changes(q):
    """
```

```
    系数序列中的符号变化次数（忽略零）。
    Descartes 规则：V(q) = #{正实根，计重数} + 2m, m >= 0。
    因此 V(q) == 0 是"q 没有正实根"的"证明"。O(n)。
    """
    signs = [1 if c > 0.0 else (-1 if c < 0.0 else 0) for c in q]
    signs = [s for s in signs if s != 0]
    return sum(1 for a, b in zip(signs, signs[1:]) if a != b)


def _reflect(q):
    """返回 q(-x) 的系数：翻转奇次项的符号。O(n)。"""
    return [c if i % 2 == 0 else -c for i, c in enumerate(q)]


def _bisect(coeffs, a, b, tol, scale, max_iter=200):
    """
    在存在符号变化的括号上进行二分。返回最终括号的中点，
    若端点不构成有效括号则返回 None。O(n * max_iter)。
    """
    fa = poly_eval(coeffs, a)
    fb = poly_eval(coeffs, b)
    # 端点本身就是根的情形
    if fa == 0.0:
        return a
    if fb == 0.0:
        return b
    # 两端同号，不能二分
    if fa * fb > 0.0:
        return None
    for _ in range(max_iter):
        m = 0.5 * (a + b)
        fm = poly_eval(coeffs, m)
        # 中点为根，或区间已足够小
        if fm == 0.0 or abs(b - a) <= tol * max(1.0, abs(m)):
            return m
        # 根据符号决定保留哪一半
        if fa * fm < 0.0:
            b, fb = m, fm
        else:
            a, fa = m, fm
    # 迭代次数用尽，返回最终中点
    return 0.5 * (a + b)


def _sample_probe(coeffs, B, tol, n_samples=16):
    """
    廉价的完整性兜底检查，每次调用 O(n)。

    在 [-B, B] 上采样 n_samples 个点；第一个符号变化就是
    （由介值定理给出的）该处存在实根的"构造性证明"，
    随后用 _bisect 把根求出来。找不到符号变化时返回 None——
    这"不是"不存在的证明，只是说廉价探测没有找到证据。
    """
    if n_samples < 2:
        return None
    scale = _scale(coeffs)
    grid = [-B + 2.0 * B * i / (n_samples - 1) for i in range(n_samples)]
    vals = [poly_eval(coeffs, x) for x in grid]
    for i in range(n_samples - 1):
```

```
        a, b = grid[i], grid[i + 1]
        fa, fb = vals[i], vals[i + 1]
        # 采样点本身是根
        if fa == 0.0:
            return a
        if fb == 0.0:
            return b
        # 相邻点符号变化 => 存在根，用二分求出来
        if fa * fb < 0.0:
            root = _bisect(coeffs, a, b, tol, scale)
            if root is not None:
                return root
    return None


def _make_result(roots, status, warnings, certified, diagnostics):
    """向后兼容的返回：默认返回列表，若要求诊断则返回 RootResult。"""
    if diagnostics:
        return RootResult(list(roots), status, tuple(warnings), certified)
    return roots


def poly_eval(coeffs, x):
    """用 Horner 格式在 x 处求多项式值。时间 O(n)。"""
    result = 0.0
    for a in reversed(coeffs):
        result = result * x + a
    return result


def poly_eval_with_deriv(coeffs, x):
    """一次 Horner 扫描同时返回 (p(x), p'(x))。时间 O(n)，空间 O(1)。"""
    p = 0.0   # p 的当前值
    dp = 0.0  # p' 的当前值
    for a in reversed(coeffs):
        dp = dp * x + p
        p = p * x + a
    return p, dp


def _eval_p_dp_ddp(coeffs, x):
    """一次 Horner 扫描同时返回 (p, p', p'')。时间 O(n)，空间 O(1)。"""
    p = 0.0
    dp = 0.0
    ddp = 0.0
    for a in reversed(coeffs):
        ddp = ddp * x + dp
        dp = dp * x + p
        p = p * x + a
    return p, dp, ddp


def _deflate(coeffs, r):
    """用 (x - r) 做合成除法；返回商多项式系数。"""
    n = len(coeffs) - 1
    quotient = [0.0] * n
    b = coeffs[n]  # 从最高次项开始
    for i in range(n - 1, -1, -1):
        quotient[i] = b
```

```
        b = coeffs[i] + b * r
    return quotient


def _deflate_power(coeffs, r, m):
    """除以 (x - r)**m；O(m * n)。"""
    q = list(coeffs)
    # 连续做 m 次单次降次
    for _ in range(m):
        q = _deflate(q, r)
    return q


def _scale(coeffs):
    """残差阈值用的归一化因子。"""
    return max(1.0, max(abs(c) for c in coeffs))


def _fujiwara_bound(coeffs):
    """
    Fujiwara 根界：每个根 r 都满足
        |r| <= 2 * max_{1<=i<=n} |a_{n-i}/a_n|^(1/i)。
    既用于流的 x_bound，也用于分层起点的构造，
    使可达区间随多项式缩放，而不随 x0 缩放。
    """
    n = len(coeffs) - 1
    if n < 1:
        return 1.0
    lead = coeffs[-1]  # 首项系数
    if lead == 0.0:
        return 1.0
    B = 0.0
    for i in range(1, n + 1):
        a = coeffs[n - i]
        if a != 0.0:
            B = max(B, abs(a / lead) ** (1.0 / i))
    return 2.0 * B


def _build_starts(B, count=7):
    """
    分层起点：一小撮固定点（0, ±1, ±0.5）廉价地覆盖常见情形，
    再加上 `count` 个在 [-B, B] 上均匀分布的点，后者随多项式的
    根界缩放，而非随 x0 缩放。去重并裁剪到 [-1.01*B, 1.01*B]。
    """
    B = max(1.0, float(B))
    count = max(1, int(count))
    # 只要一个起点时直接返回 0
    if count == 1:
        return [0.0]
    # 均匀分布点
    pts = [-B + 2.0 * B * i / (count - 1) for i in range(count)]
    extras = [0.0, 1.0, -1.0, 0.5, -0.5]
    out, seen = [], set()
    # 先放固定点，再放均匀点；去重并裁剪范围
    for s in extras + pts:
        s = float(s)
        key = round(s, 12)
        if key in seen or abs(s) > B * 1.01:
```

```
            continue
        seen.add(key)
        out.append(s)
    return out


def _flow_root(coeffs, x0, tol=1e-12, max_iter=400):
    """
    GCNMTr 的标量特化版本。

    保留自 GCNMTr 的部分：
      - rho_k = (|F_k|-|F_{k+1}|) / (|F_k|-|F_k+J_k s_k|)
      - 三区间 dt 更新：gamma1=2, eta1=0.25, eta2=0.75, gamma2=0.5
      - 当 rho_k >= eta_a = 1e-6 时接受，否则回滚 x
    标量偏离（已记录，非缺陷）：
      - J = p' 由 Horner 得到；J^T 的 QR 分解平凡（R=|p'|, Q=sign(p')）
      - Levenberg lam 是 p'=0 处的数值保护；它使 s=0（流停滞），
        这是连续牛顿在"非根驻点"处的正确行为，而非为 p'=0 做的修补。
    边界用 Fujiwara 界（非基于 x0），使大根可达。
    返回 (x, reason)，reason 属于
    {converged, stationary, diverged, dt_underflow, near_miss, max_iter}。
    """
    eta_a, eta_1, eta_2 = 1e-6, 0.25, 0.75
    gamma_1, gamma_2 = 2.0, 0.5
    dt = 1.0                    # 标量情形：整步
    lam = 1e-10                 # Levenberg 正则项初值
    x = float(x0)
    scale = _scale(coeffs)
    x_bound = 10.0 * max(1.0, _fujiwara_bound(coeffs))
    p0, dp0 = poly_eval_with_deriv(coeffs, x)
    r0 = abs(p0)
    if r0 <= tol * scale:
        return x, "converged"
    # 若第一步牛顿步会越界，先缩小 dt 以防止过冲
    if dp0 != 0.0:
        sN0 = -p0 / dp0
        if math.isfinite(sN0) and abs(sN0) > x_bound:
            dt = max(1e-2, x_bound / abs(sN0))

    best_x, best_r = x, r0
    for _ in range(max_iter):
        p, dp = poly_eval_with_deriv(coeffs, x)
        r = abs(p)
        # 已足够接近根
        if r <= tol * scale:
            return x, "converged"
        denom = dp * dp + lam
        if denom == 0.0:
            return best_x, "stationary"
        sN = -p * dp / denom
        s = dt / (1.0 + dt) * sN
        if s == 0.0:
            if dp == 0.0:
                return best_x, "stationary"
            return best_x, "dt_underflow"
        x_new = x + s
        # 越界或非有限 => 发散
        if not math.isfinite(x_new) or abs(x_new) > x_bound:
            return best_x, "diverged"
```

```
        p_new = poly_eval(coeffs, x_new)
        r_new = abs(p_new)
        actual = r - r_new                      # 实际下降量
        pred = r - abs(p + dp * s)              # 线性模型预测下降量
        # 只有实际下降与预测下降都为正时才定义比值
        rho = actual / pred if (actual > 0.0 and pred > 0.0) else -1.0
        if abs(1.0 - rho) < eta_1:
            dt = gamma_1 * dt          # 比值很好：放大步长
        elif abs(1.0 - rho) < eta_2:
            pass                        # 比值尚可：保持步长
        else:
            dt = gamma_2 * dt           # 比值差：缩小步长
        # 限制步长上限，防止数值爆炸
        if dt > 1e6:
            dt = 1e6

        # --- 接受 / 拒绝 ---
        if rho >= eta_a:
            x, r = x_new, r_new
            if r_new < best_r:
                best_x, best_r = x_new, r_new
            lam = max(lam * 0.3, 1e-14)
            # 即使因舍入使 actual <= 0，只要已到根附近就返回
            if r_new <= tol * scale:
                return x_new, "converged"
        else:
            lam = min(lam * 10.0, 1.0)
            if dt < 1e-15:
                return best_x, "dt_underflow"
    # 迭代次数用尽：做最后判断
    if best_r <= max(1e-8, tol * scale):
        return best_x, "converged"
    if best_r < 0.5 * r0:                # 确有进展，精修可能救回
        return best_x, "near_miss"
    return best_x, "max_iter"


def _polish_root(coeffs, x, tol=1e-13, max_iter=50):
    """
    在"原始"多项式上精修一个根。

    第一阶段是原求解器的普通牛顿迭代（因此单根精度完全不变）。
    第二阶段仅在重数估计接近整数 m0 >= 2 且 |p| 安全地高于舍入
    下限时进入，使用修正步
    x <- x - m0 * p / p'  （重根下二次收敛）。
    """
    xx = float(x)
    # ---- 第一阶段：带回溯的牛顿迭代（绝不跳到另一个根）----
    # 只有在 |p| 下降时才走纯牛顿步；当流已停在舍入下限时，
    # 不接受任何步，保留当前点（跳跃会污染降次）。
    best_x = xx
    best_r = abs(poly_eval(coeffs, xx))
    for _ in range(max_iter):
        f, df = poly_eval_with_deriv(coeffs, xx)
        if df == 0.0:
            break
        step = f / df
        # 步长非有限或过大则终止
        if not math.isfinite(step) or abs(step) > 10.0 * max(1.0, abs(xx)):
```

```
            break
        lam = 1.0
        xn = xx - step
        rn = abs(poly_eval(coeffs, xn))
        tries = 0
        # 回溯：不断折半步长，直到残差下降
        while rn >= best_r and tries < 40:
            lam *= 0.5
            tries += 1
            xn = xx - lam * step
            rn = abs(poly_eval(coeffs, xn))
        if rn >= best_r:
            break
        xx = xn
        best_x, best_r = xn, rn
        # 步长已足够小，收敛
        if abs(lam * step) <= tol * max(1.0, abs(xx)):
            break

    # ---- 第二阶段：重数加速精修（用于重根）----
    scale = _scale(coeffs)
    floor = 1e3 * _EPS * scale
    p, dp, ddp = _eval_p_dp_ddp(coeffs, xx)
    m0 = None
    # 只在导数非零且残差高于舍入下限时估计重数
    if dp != 0.0 and abs(p) > floor:
        den = dp * dp - 2.0 * p * ddp
        if den > 0.0:
            mh = dp * dp / den
            k = round(mh)
            # 估计值接近整数 k（>=2）才采用修正步
            if 2 <= k <= 500 and abs(mh - k) <= 0.02 * k:
                m0 = k
    if m0 is None:
        return xx

    best_x, best_r = xx, abs(p)
    for _ in range(60):
        p, dp, ddp = _eval_p_dp_ddp(coeffs, xx)
        if dp == 0.0 or not math.isfinite(p):
            break
        step = m0 * p / dp
        if not math.isfinite(step) or abs(step) > 10.0 * max(1.0, abs(xx)):
            break
        xn = xx - step
        if not math.isfinite(xn):
            break
        rn = abs(poly_eval(coeffs, xn))
        # 残差明显上升则终止，避免跳到别处
        if rn > 10.0 * best_r + floor:
            break
        xx = xn
        if rn < best_r:
            best_x, best_r = xn, rn
        # 步长已足够小，收敛
        if abs(step) <= tol * max(1.0, abs(xx)):
            break
    return best_x
```

```
# ---------------------------------------------------------------- Decimal

def _dec_eval3(cd, x):
    """Decimal 版本的 Horner：同时返回 (p, p', p'')。"""
    p = Decimal(0)
    dp = Decimal(0)
    ddp = Decimal(0)
    for a in reversed(cd):
        ddp = ddp * x + dp
        dp = dp * x + p
        p = p * x + a
    return p, dp, ddp


def _dec_eval(cd, x):
    """Decimal 版本的 Horner 求值。"""
    p = Decimal(0)
    for a in reversed(cd):
        p = p * x + a
    return p


def _certify_at(coeffs, r, max_mult, digits):
    """
    在指定精度下认证根位置与重数。
    通过局部对数斜率（相邻点函数值之比的对数）估计重数。
    """
    with localcontext() as ctx:
        ctx.prec = digits                           # 设定工作精度
        cd = [Decimal(c) for c in coeffs]           # 系数转 Decimal
        n = len(cd) - 1
        if n < 1:
            return (float(r), None, False)
        tiny = Decimal(10) ** (-(digits - 15))
        x = Decimal(r)
        best_x, best_p = x, abs(_dec_eval(cd, x))

        # ---- 带回溯的牛顿迭代，且考虑重数 ----
        for _ in range(150):
            p, dp, ddp = _dec_eval3(cd, x)
            if dp == 0:
                break
            mult = Decimal(1)                       # 默认按单根处理
            den = dp * dp - 2 * p * ddp
            if den > 0:
                mh = dp * dp / den
                m0 = int(mh.to_integral_value(rounding=ROUND_HALF_EVEN))
                # 若接近整数（>=2）则用修正步
                if 2 <= m0 <= 2000 and abs(mh - m0) <= Decimal("0.02") * m0:
                    mult = Decimal(m0)
            step = mult * p / dp
            lam = Decimal(1)
            xn = x - step
            rn = abs(_dec_eval(cd, xn)) if xn.is_finite() else None
            tries = 0
            # 回溯，直到残差下降
            while rn is None or rn > best_p:
                lam /= 2
```

```
            tries += 1
            if tries > 30:
                break
            xn = x - lam * step
            rn = abs(_dec_eval(cd, xn)) if xn.is_finite() else None
        if rn is None or rn >= best_p:
            break                       # 无法继续下降
        best_x, best_p = xn, rn
        x = xn
        if abs(step * lam) <= tiny * max(Decimal(1), abs(x)):
            break
    x = best_x

    # ---- 通过局部对数斜率确定重数 ----
    base = max(Decimal(1), abs(x))
    scale = Decimal(0)                      # 用于估计噪声水平
    pw = Decimal(1)
    for i in range(n + 1):
        scale += abs(cd[i]) * pw
        pw *= base
    noise = scale * Decimal(10) ** (-(digits - 8))
    ln2 = Decimal(2).ln()
    first_slope = None
    m_bound = max(1, min(max_mult, n))
    last = None
    run = 0                                 # 连续一致估计的计数
    m_found = None
    # 在 8 个不同 eta 尺度下探测（base*1e-9 .. base*1e-2）
    for k in range(0, 8):
        eta = base * Decimal(10) ** (-9 + k)
        p1 = _dec_eval(cd, x + eta)
        p2 = _dec_eval(cd, x + 2 * eta)
        if p1 == 0 or p2 == 0:
            run = 0
            continue
        a1, a2 = abs(p1), abs(p2)
        # 低于噪声水平的样本不可信
        if a1 < noise or a2 < noise:
            run = 0
            continue
        slope = (a2 / a1).ln() / ln2
        if first_slope is None:
            first_slope = slope
        cand = int(slope.to_integral_value(rounding=ROUND_HALF_EVEN))
        # 接近整数且在界内则视为候选
        if 1 <= cand <= m_bound and abs(slope - cand) <= Decimal("0.02") * cand:
            if cand == last:
                run += 1
            else:
                last, run = cand, 1
            # 连续两次一致才确认
            if run >= 2:
                m_found = cand
                break
        else:
            last, run = None, 0
    if m_found is None:
        # 样本良好但局部斜率 ~ 0：该点不是根
        if first_slope is not None and abs(first_slope) < Decimal("0.4"):
```

```
                return (float(x), 0, False)
            return (float(x), None, False)
        return (float(x), m_found, True)


def _certify_root(coeffs, r, max_mult):
    """
    高精度下认证 (位置, 重数)，并自适应工作精度。
    返回 (r_float, m, ok)。
    """
    # 依次尝试 1x, 2x, 4x 默认精度，直到认证成功
    for digits in (_CERT_DIGITS, 2 * _CERT_DIGITS, 4 * _CERT_DIGITS):
        r_c, m_c, ok = _certify_at(coeffs, r, max_mult, digits)
        if ok:
            return (r_c, m_c, True)
        if m_c == 0:                        # 被认证为"不是根"
            return (r_c, 0, False)
    return (float(r), None, False)


# ---------------------------------------------------------------- canonical

def _canonicalize(entries, coeffs, allow_certify, cluster_tol=None):
    """
    合并代表同一个（数值上被拆开的）根的条目。

    返回 (merged_entries, merge_warnings)。一个足够接近、应当合并、
    但其合并后的重数无法认证的簇，"不会"在没有提示的情况下被
    静默保留为拆开的条目：保留原始条目并发出警告。
    """
    if len(entries) <= 1:
        return sorted(entries, key=lambda t: t[0]), []

    def delta_of(m, center):
        """根据重数 m 与中心位置给出合并容差。"""
        if cluster_tol is not None:
            return float(cluster_tol)
        if allow_certify:
            return min(_CANON_C * _EPS ** (1.0 / m), _CANON_CAP) * max(1.0, abs(center))
        return 1e-9 * max(1.0, abs(center))

    srt = sorted(entries, key=lambda t: t[0])
    # 按位置排序后，贪心地构造簇
    groups = [[srt[0]]]
    for e in srt[1:]:
        g = groups[-1]
        tot = sum(mm for _, mm in g)
        center = sum(rr * mm for rr, mm in g) / tot
        m_tot = tot + e[1]
        delta = delta_of(m_tot, center)
        # 若与当前簇中心足够接近则并入
        if abs(e[0] - center) <= delta:
            g.append(e)
        else:
            groups.append([e])

    out = []
    merge_warnings = []
    for g in groups:
```

```
        # 单元素簇直接保留
        if len(g) == 1:
            out.append(g[0])
            continue
        tot = sum(mm for _, mm in g)
        center = sum(rr * mm for rr, mm in g) / tot
        accepted = False
        # 尝试用 Decimal 认证合并后的 (位置, 重数)
        if allow_certify and coeffs is not None:
            r_c, m_c, ok = _certify_root(coeffs, center, len(coeffs) - 1)
            if (ok and m_c is not None and 1 <= m_c <= len(coeffs) - 1
                    and abs(r_c - center) <= 5.0 * delta_of(m_tot, center)):
                out.append((r_c, m_c))
                accepted = True
        # 认证失败：不静默合并，保留原始条目并发出警告
        if not accepted:
            merge_warnings.append(
                "cluster near r=%r (sum m=%d) not merged: "
                "certification failed; original entries kept" % (center, tot))
            out.extend(g)
    out.sort(key=lambda t: t[0])
    return out, merge_warnings


def poly_real_roots(coeffs, tol=1e-12, *, canonical=True,
                    cluster_tol=None, precision="auto", raw=False,
                    diagnostics=False, max_degree=None,
                    completeness="fast", strict=False):
    # 参数合法性检查：precision 只能是 auto/double/high
    if precision not in ("auto", "double", "high"):
        raise ValueError("precision must be 'auto', 'double' or 'high'")
    # 参数合法性检查：completeness 只能是 fast/verified
    if completeness not in ("fast", "verified"):
        raise ValueError("completeness must be 'fast' or 'verified'")
    # raw=True 时强制关闭规范化输出
    if raw:
        canonical = False

    # ---- input guard ------------------------------------------------
    # 输入保护：coeffs 为 None 时按 strict 决定抛错还是返回错误结果
    if coeffs is None:
        if strict:
            raise ValueError("coeffs is None")
        return _make_result([], "input_error", ("coeffs is None",),
                            False, diagnostics)
    # 把系数统一转换成 float
    try:
        a = [float(c) for c in coeffs]
    except OverflowError:
        # 系数超出 double 表示范围
        if strict:
            raise
        return _make_result(
            [], "overflow_risk",
            ("coefficient exceeds double range; pass factored form "
             "or reduce degree",), False, diagnostics)
    except (TypeError, ValueError) as e:
        # 存在非数值系数
        if strict:
```

```
            raise ValueError("non-numeric coefficient: %s" % e)
        return _make_result([], "input_error",
                            ("non-numeric coefficient: %s" % e,),
                            False, diagnostics)
    # 逐个检查系数是否为有限值（排除 nan/inf）
    for i, c in enumerate(a):
        if not math.isfinite(c):
            if strict:
                raise ValueError("non-finite coefficient at index %d" % i)
            return _make_result([], "input_error",
                                ("non-finite coefficient at index %d" % i,),
                                False, diagnostics)
    # 零多项式：所有实数都是根，无法给出有限根表
    if all(c == 0.0 for c in a):
        if strict:
            raise ValueError("zero polynomial: every real is a root")
        return _make_result([], "input_error",
                            ("zero polynomial: every real is a root",),
                            False, diagnostics)

    # ---- preprocessing (unchanged) ------------------------------------
    # 去掉末尾的高阶零系数（即降低名义次数）
    hi = len(a)
    while hi > 1 and a[hi - 1] == 0.0:
        hi -= 1
    a = a[:hi]
    lo = 0
    while lo < len(a) - 1 and a[lo] == 0.0:
        lo += 1
    mult_zero = lo
    a = a[lo:]
    n = len(a) - 1
    # 次数超过用户设定的上限时直接返回
    if max_degree is not None and n > max_degree:
        if strict:
            raise ValueError("degree %d exceeds max_degree %d"
                             % (n, max_degree))
        return _make_result(
            [], "degree_too_high",
            ("degree %d exceeds max_degree %d; O(n^2 k) expected"
             % (n, max_degree),), False, diagnostics)
    # 常数多项式：只有零根（若前面统计到重数）
    if n < 1:                       # constant polynomial
        roots = [(0.0, mult_zero)] if mult_zero else []
        return _make_result(roots, "ok", (), True, diagnostics)
    lead = a[-1]
    if lead != 1.0:
        a = [c / lead for c in a]
    original = list(a)
    roots = [(0.0, mult_zero)] if mult_zero else []
    q = list(a)
    certify_ok = ((precision != "double")
                  and (len(original) - 1 <= _CERT_MAX_DEG))
    certified = True
    warnings_list = []
    unverified = False

    # ---- main loop ----------------------------------------------------
    # 主循环：只要工作多项式次数 >= 2 就继续找根
```

```
    while len(q) - 1 >= 2:
        # Layer 1: Descartes gate, O(n).  V+ == V- == 0 is a proof.
        # 第一层：Descartes 符号变化数门控，O(n)
        V_pos = _descartes_changes(q)          # 正根符号变化数上界
        V_neg = _descartes_changes(_reflect(q))  # 负根（q(-x)）符号变化数上界
        # 正负符号变化数都为 0 且常数项非零，则证明已无实根
        if V_pos == 0 and V_neg == 0 and q[0] != 0.0:
            if _DEBUG:
                print("[descartes] certified no real roots remain")
            break

        # Layer 2: adaptive starts, O(1) factor.
        # 第二层：自适应起点数，依据 Fujiwara 界 B 构造起点
        B = max(1.0, _fujiwara_bound(q))
        n_starts = max(7, min(V_pos + V_neg, 32))
        starts = _build_starts(B, n_starts)
        found = None
        near_miss = None
        for s in starts:
            r_cand, reason = _flow_root(q, s, tol)
            if _DEBUG:
                print("[flow] start=%r reason=%s x=%r" % (s, reason, r_cand))
            # 成功收敛，直接采用
            if reason == "converged":
                found = r_cand
                break
            # 近似命中，先记录第一个，后续再抛光验证
            if reason == "near_miss" and near_miss is None:
                near_miss = r_cand
        # 若没有收敛结果但有近似命中，则用原始多项式抛光并校验残差
        if found is None and near_miss is not None:
            r_pol = _polish_root(original, near_miss)
            if abs(poly_eval(original, r_pol)) <= 1e-8 * _scale(original):
                found = r_pol
            elif _DEBUG:
                print("[flow] near_miss rejected: |p|=%.3e"
                      % abs(poly_eval(original, r_pol)))

        # Layer 3: sample probe, O(n) per round.
        # 第三层：若流式方法全部失败，则在 [-B, B] 上采样探测变号
        if found is None:
            probe = _sample_probe(q, B, tol)
            if probe is not None:
                found = probe
                warnings_list.append(
                    "flow stalled (V+=%d, V-=%d); root recovered "
                    "by sample probe" % (V_pos, V_neg))
            else:
                unverified = True
                warnings_list.append(
                    "flow stalled at all %d starts (V+=%d, V-=%d); "
                    "sample probe found no sign change; remaining "
                    "roots unverified" % (len(starts), V_pos, V_neg))
                break
        r = _polish_root(original, found)
        if _DEBUG:
            print("[loop] flow=%r polished=%r |p|=%.3e"
                  % (found, r, abs(poly_eval(q, r))))
        q_save = q
```

```
        q = _deflate(q, r)
        m = 1
        qs = _scale(q)
        # 若 r 仍是收缩后多项式的近似根，则继续累加重数
        while len(q) > 1 and abs(poly_eval(q, r)) < 1e-6 * qs:
            q = _deflate(q, r)
            m += 1
            qs = _scale(q)
        rho = (abs(poly_eval(q, r)) / qs
               if (len(q) > 1 and qs) else math.inf)

        # 在需要且允许的情况下，对重数做认证
        if certify_ok and (m >= 2 or rho < 1e-2
                           or not math.isfinite(rho)):
            r_c, m_c, ok_c = _certify_root(original, r, len(original) - 1)
            if _DEBUG:
                print("[cert] r_c=%r m_c=%s ok=%s" % (r_c, m_c, ok_c))
            # 认证结果合理则采纳认证后的根与重数，并重新收缩
            if (ok_c and m_c is not None
                    and 1 <= m_c <= len(q_save) - 1
                    and abs(r_c - r) <= 0.1 * max(1.0, abs(r))
                    and m_c <= 2 * m + 2):
                r, m = r_c, m_c
                q = _deflate_power(q_save, r, m)
            elif m >= 2:
                # 认证失败但重数 >= 2：标记为未完全认证
                certified = False
                warnings_list.append(
                    "multiplicity of root %r not certified" % r)
        elif m >= 2 and not certify_ok:
            # 不满足认证条件但重数 >= 2：同样标记为未完全认证
            certified = False
            warnings_list.append(
                "multiplicity of root %r not certified "
                "(degree > %d or precision='double')"
                % (r, _CERT_MAX_DEG))
        roots.append((r, m))

    # ---- linear remainder ------------------------------------------
    # 剩余一次多项式：直接用 -b/a 求根并抛光
    if len(q) - 1 == 1:
        r = _polish_root(original, -q[0] / q[1])
        roots.append((r, 1))

    # ---- canonical output (warning-aware) --------------------------
    # 规范化输出：合并近似重复根、排序，并收集相关警告
    if canonical:
        roots, merge_warns = _canonicalize(
            roots, original, certify_ok, cluster_tol)
        if merge_warns:
            warnings_list.extend(merge_warns)
            certified = False
    else:
        roots.sort(key=lambda item: item[0])

    # ---- status ----------------------------------------------------
    # 根据是否未验证 / 是否完全认证决定最终状态
    if unverified:
        status = "unverified"
```

```
    elif not certified:
        status = "partial"
    else:
        status = "ok"
    return _make_result(roots, status, tuple(warnings_list),
                        certified, diagnostics)
```

代码 7 solve_p5.py

```
"""
solve_p5.py
===========
使用 poly_roots.py 中的通用求解器求解

    p5(x) = x^5 - 6x^3 - x^2 - 5x + 5

然后通过以下方式验证结果：
(1) 逐根残差（residual）
(2) 韦达定理（Vieta's relations）
(3) numpy.roots 作为独立交叉验证
"""

from poly_roots import poly_real_roots, poly_eval

# 系数按升幂排列：5 - 5x - x^2 - 6x^3 + 0*x^4 + x^5
COEFFS = [5.0, -5.0, -1.0, -6.0, 0.0, 1.0]


def main():
    # 调用通用多项式实根求解器，容差设为 1e-12
    # 返回值为 (根, 重数) 的列表
    roots = poly_real_roots(COEFFS, tol=1e-12)

    print("p5(x) = x^5 - 6x^3 - x^2 - 5x + 5")
    print("=" * 52)
    print("real roots found: %d (degree 5)" % len(roots))  # 找到的实根个数（次数为 5）
    print("-" * 52)
    # 表头：# 编号、root 根、mult 重数、residual 残差
    print("%4s  %20s  %4s  %12s" % ("#", "root", "mult", "residual"))
    for i, (r, m) in enumerate(roots, 1):
        # 对每个根计算 |p5(r)| 作为残差，检验其是否接近 0
        print("%4d  %20.12f  %4d  %12.3e" % (i, r, m, abs(poly_eval(COEFFS, r))))
    print("-" * 52)

    # 韦达定理：所有根（含重数）之和必须等于 -a4/a5
    # 本多项式中 a4 = 0, a5 = 1，所以目标值为 0
    total = sum(r * m for r, m in roots)
    print("Vieta check: sum of real roots = %.15f (target -a4/a5 = 0)" % total)
    # 如果和不为零，说明缺失的复根对的实部不为零
    print("(a nonzero sum indicates the missing complex pair's real part)")

    try:
        import numpy as np
        # numpy.roots 要求系数按降幂排列，因此用 COEFFS[::-1] 反转
        all_roots = np.roots(COEFFS[::-1])
        # 只保留虚部接近 0 的根，视为实根，并排序
```

```
        real_ref = sorted(r.real for r in all_roots if abs(r.imag) < 1e-9)
        print("-" * 52)
        print("numpy.roots cross-check (real roots):")
        for r in real_ref:
            print("    %.12f" % r)
        # 比较我们的求解器与 numpy 得到的实根是否一致：
        # 数量相同，且逐一对齐后差值小于 1e-8
        match = len(real_ref) == len(roots) and all(
            abs(a - b) < 1e-8 for a, b in zip(real_ref, [r for r, _ in roots])
        )
        print("agreement with our solver:", match)
        # 筛选出剩余的复根对（虚部不可忽略的根）
        complex_pair = [r for r in all_roots if abs(r.imag) >= 1e-9]
        print("remaining complex pair:")
        for r in complex_pair:
            # 以 "实部 ± 虚部 i" 的形式打印复根
            print("    %.12f %+.12fi" % (r.real, r.imag))
    except ImportError:
        # 如果环境中没有 numpy，则跳过交叉验证部分
        pass


if __name__ == "__main__":
    main()
```